LEBANESE INTERNATIONAL UNIVERSITY
Department of Computer Science and Information Technology

CSCI426 · Advanced Web Programming

Project Phase 1

HealthTrack

A Fitness and Wellness Tracking Web Application



Student	Ali Mostafa
Student ID	_____
Instructor	_____
Theme	6 – HealthTrack (Fitness & Wellness)
Submitted	November 2026

Contents

1. Abstract

2. Introduction and Problem

3. System Design

4. Technologies Used

5. User Interface

6. Key Code Snippets

7. Responsive Design

8. Version Control and Deployment

9. Conclusion

1. Abstract

HealthTrack is a frontend web application built with ReactJS that helps people follow a healthy daily routine. Users can log their **meals** (in calories), their **exercise** and their **daily activities** (in minutes), and the application adds everything up into a summary of the day. HealthTrack also offers six ready **workout plans** that can be searched and filtered; each plan opens on its own dynamic page that lists its exercises and can be added to the tracker with one click.

The application has seven pages connected with React Router, a light and a dark theme, validated forms, and a responsive layout that works on desktop, tablet and mobile. Data is stored in the browser with localStorage, so no backend or account is needed. The source code is managed with Git and hosted on GitHub, and the site is deployed online.

2. Introduction and Problem

Many students and beginners want to eat better and exercise more, but they lose track: they forget what they ate, how long they trained, or which exercises to do. Most fitness apps are complicated, need an account, or are paid.

HealthTrack solves this with a simple, free website where a user can write down a meal or a workout in a few seconds and immediately see the total for today. The main goals were:

- Make logging meals, exercise and activities fast and easy.
- Show a clear daily summary (calories eaten, active minutes, number of logs).
- Give beginners ready workout plans with clear exercises, sets and reps.
- Offer a clean, modern and responsive design with a dark mode.

3. System Design

3.1 Component Architecture

The application starts in **index.js**, which renders the **App** component. App holds the theme state and contains the Navbar, the Footer and the Router. The Router shows one page component for each URL. Pages reuse small components (StatCard and WorkoutCard) and read their data from **workouts.json** and **localStorage**.

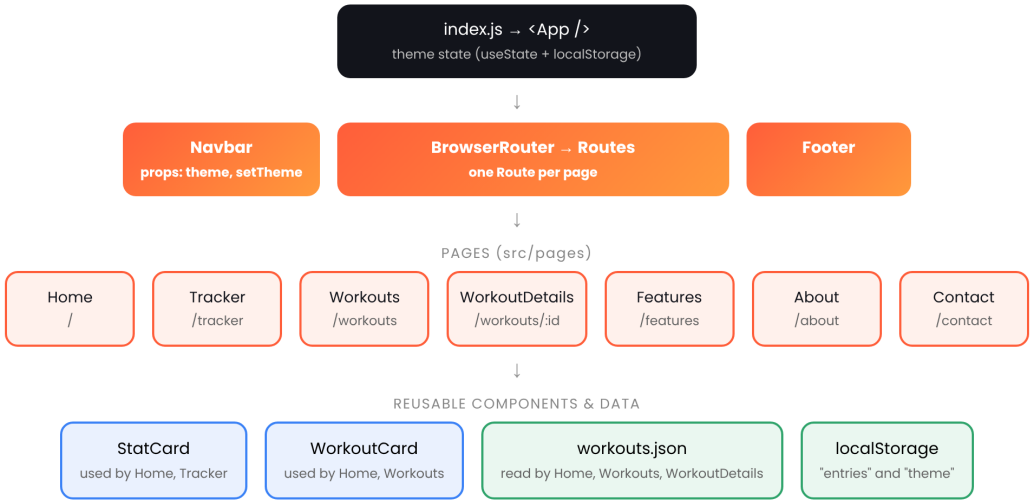


Figure 1 – Component tree of HealthTrack

3.2 Pages and Routes

Route	Page	Purpose
/	Home	Hero section, today's summary, three popular workouts, call to action
/tracker	Tracker	Validated form to add logs, list with filters and delete, daily totals
/workouts	Workouts	All workout plans with a search box and category filters
/workouts/:id	WorkoutDetails	**Dynamic page**: shows the workout whose id is in the URL
/features	Features	Six feature cards describing the app
/about	About	Mission, target users and a 3-step “how it works”
/contact	Contact	Contact details and a validated message form
*	NotFound	404 page for any wrong link

3.3 Data Model

A **log entry** (saved in localStorage under the key `entries`):

Field	Type	Example
id	number	1762889000000 (from Date.now(), unique)
type	text	"Meal", "Exercise" or "Activity"
name	text	"Chicken salad"
value	number	450 (kcal for meals, minutes for the others)
date	text	"2026-11-11"

A **workout** (in src/data/workouts.json) has an id, name, category, level, minutes, calories, a description and a list of exercises (name, sets, reps).

3.4 Data Flow when Adding a Log

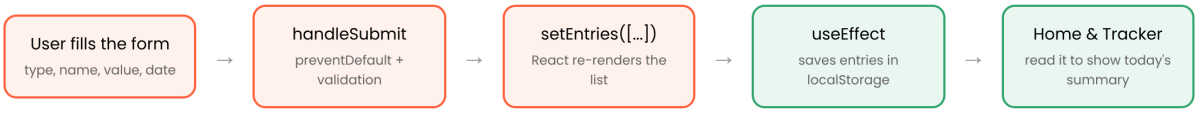


Figure 2 – What happens when the user submits the Tracker form

4. Technologies Used

Technology	Used for
ReactJS 18 (Create React App)	Building the user interface from reusable components
React Router DOM 6	BrowserRouter, Routes, Route, NavLink, Link and useParams for navigation and the dynamic page
React Hooks	useState for form inputs, filters, theme and menu; useEffect to save data
localStorage	Keeping the logs and the chosen theme after the page is refreshed
JSON	Storing the workout plans (src/data/workouts.json)
CSS3	CSS variables for the two themes, Flexbox, Grid, transitions, keyframe animation, media queries
Google Fonts (Poppins)	Modern, readable typography
Git & GitHub	Version control and commit history
Netlify / Vercel	Hosting the live website
VS Code	Code editor

5. User Interface

The design uses one dark navy color (#12141b) with an orange gradient (#ff5e3a to #ff9a3c) as the brand color. Cards have rounded corners and soft shadows, and buttons and cards lift slightly on hover. The navbar is sticky and highlights the current page.

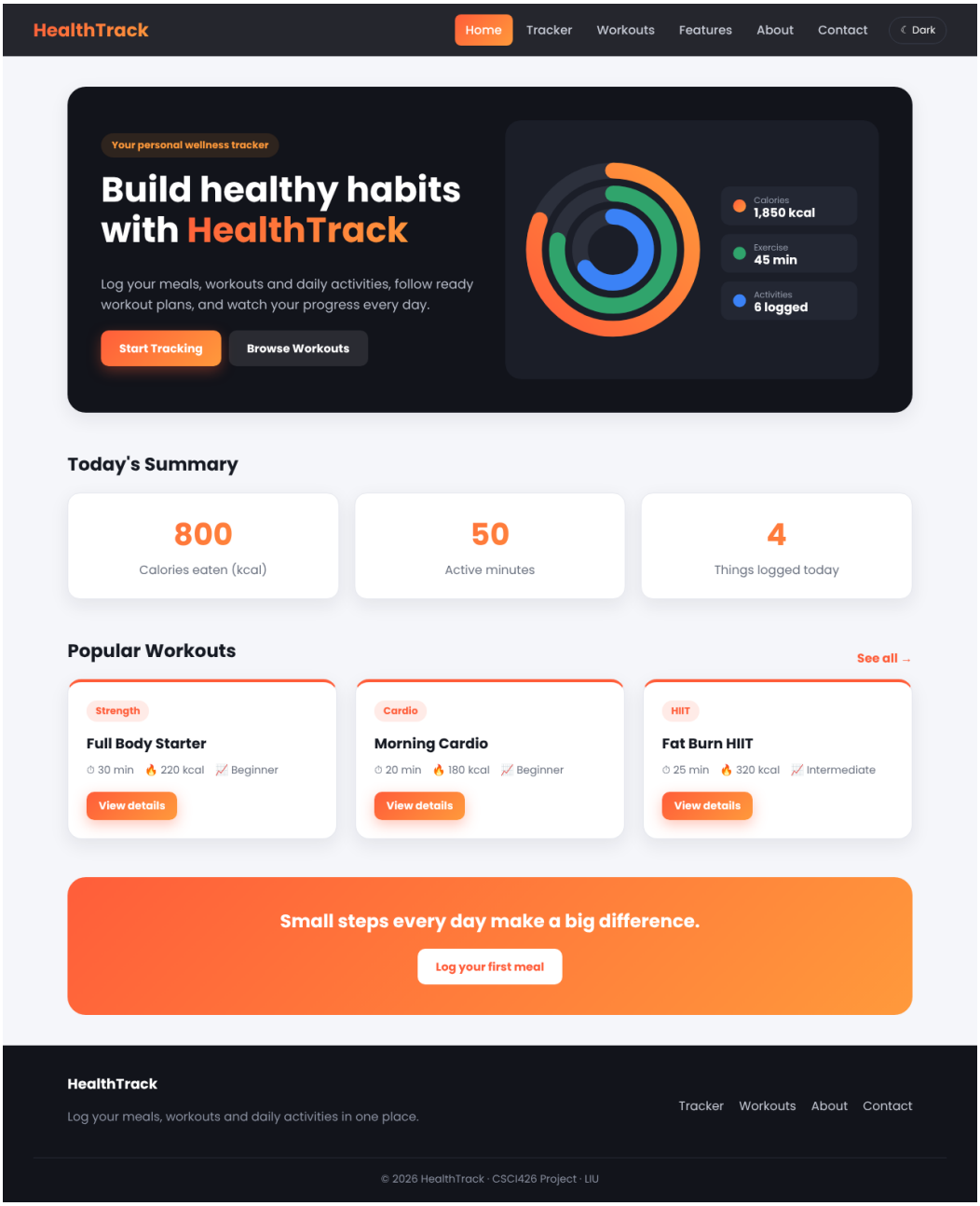


Figure 3 – Home page

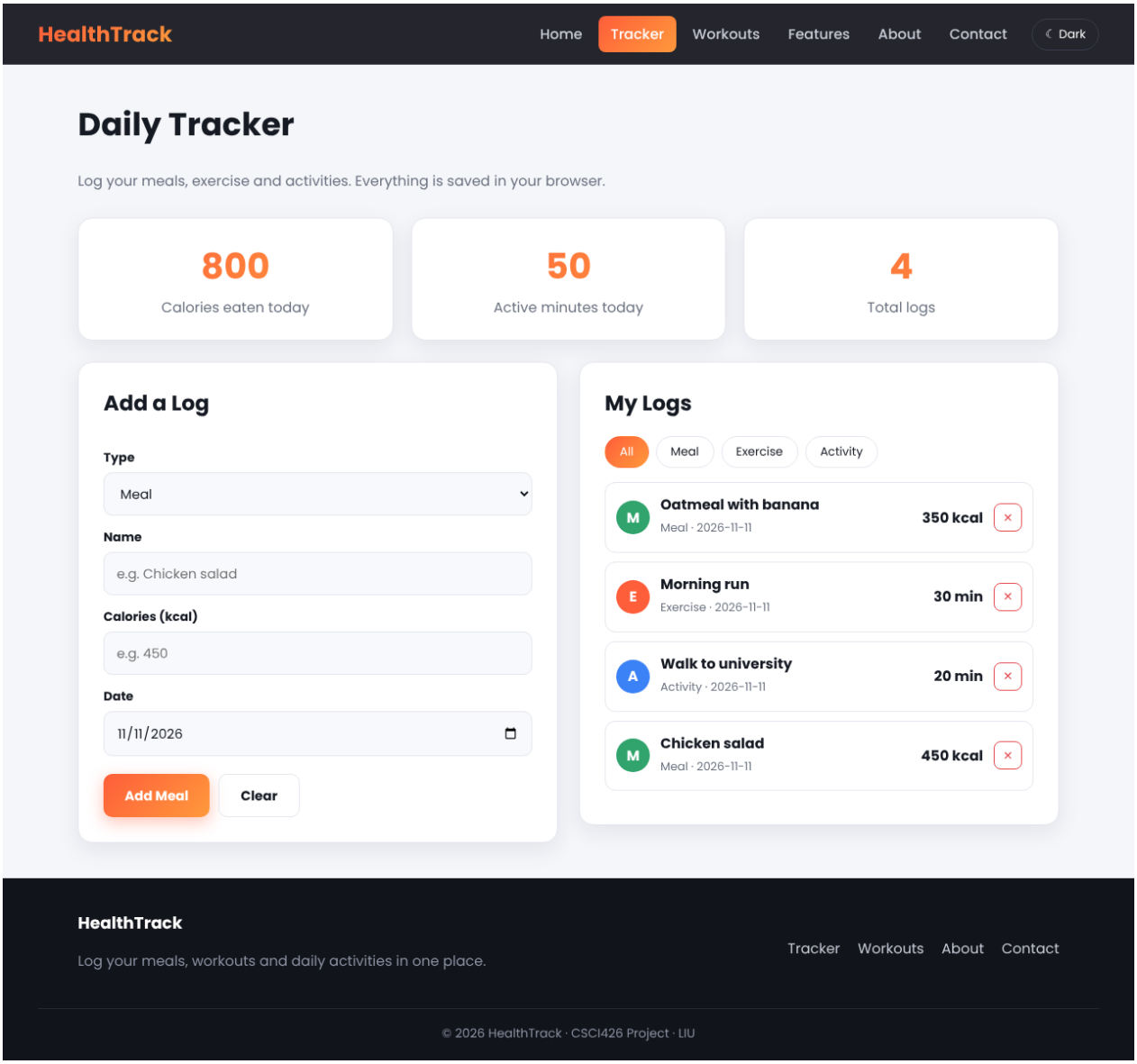


Figure 4 – Daily Tracker

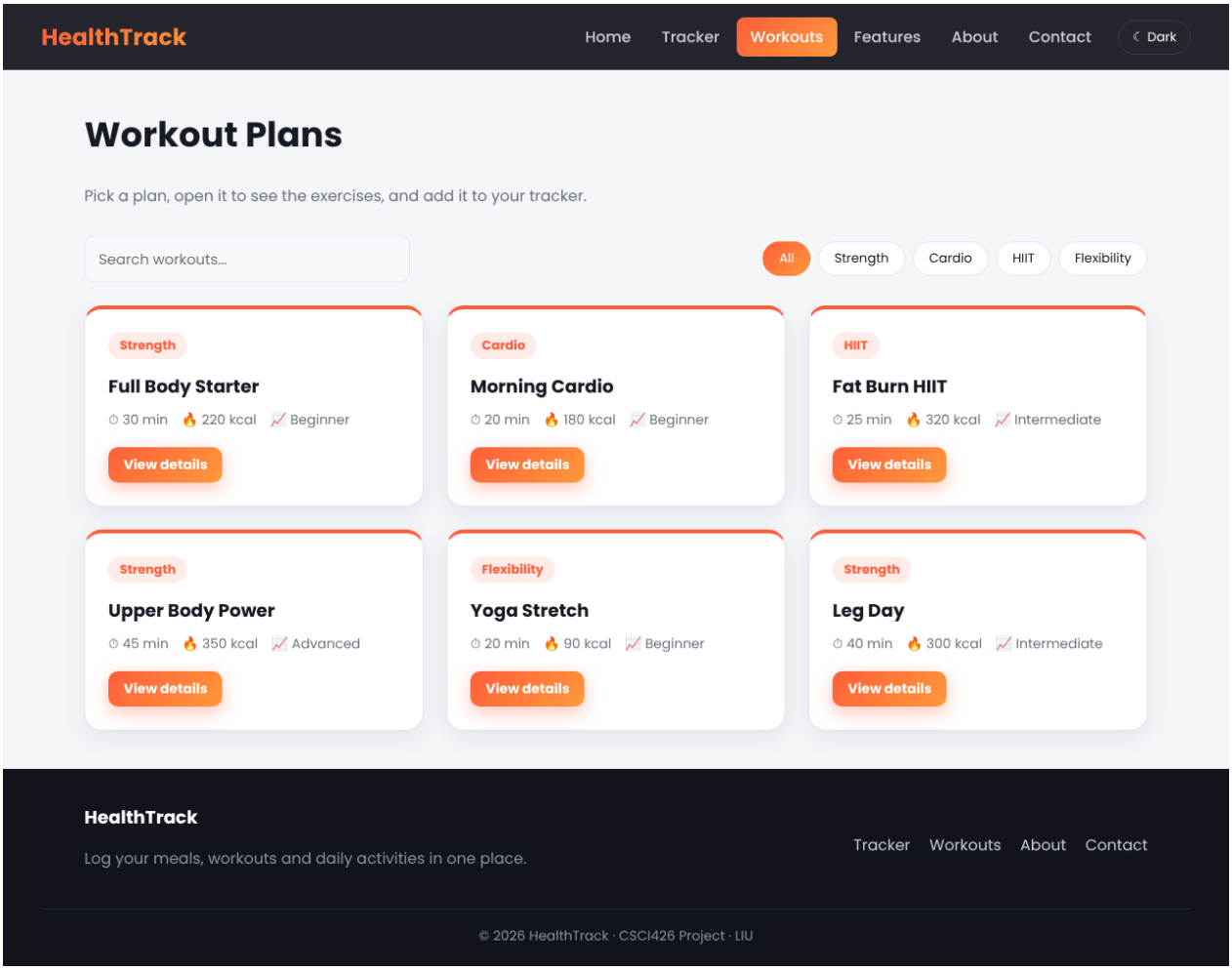


Figure 5 – Workout Plans with search and filters

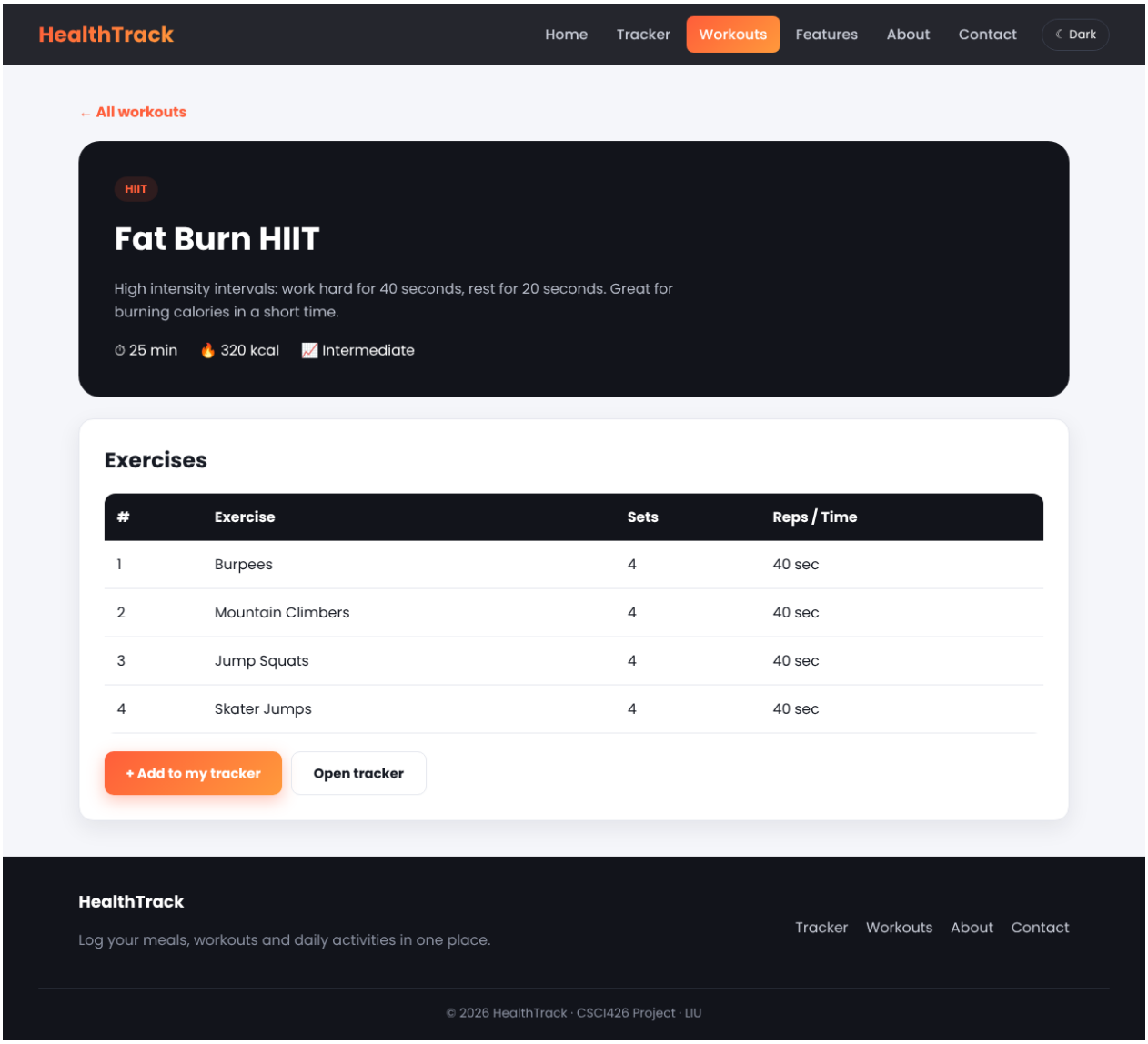


Figure 6 – Dynamic Workout Details page (/workouts/3)

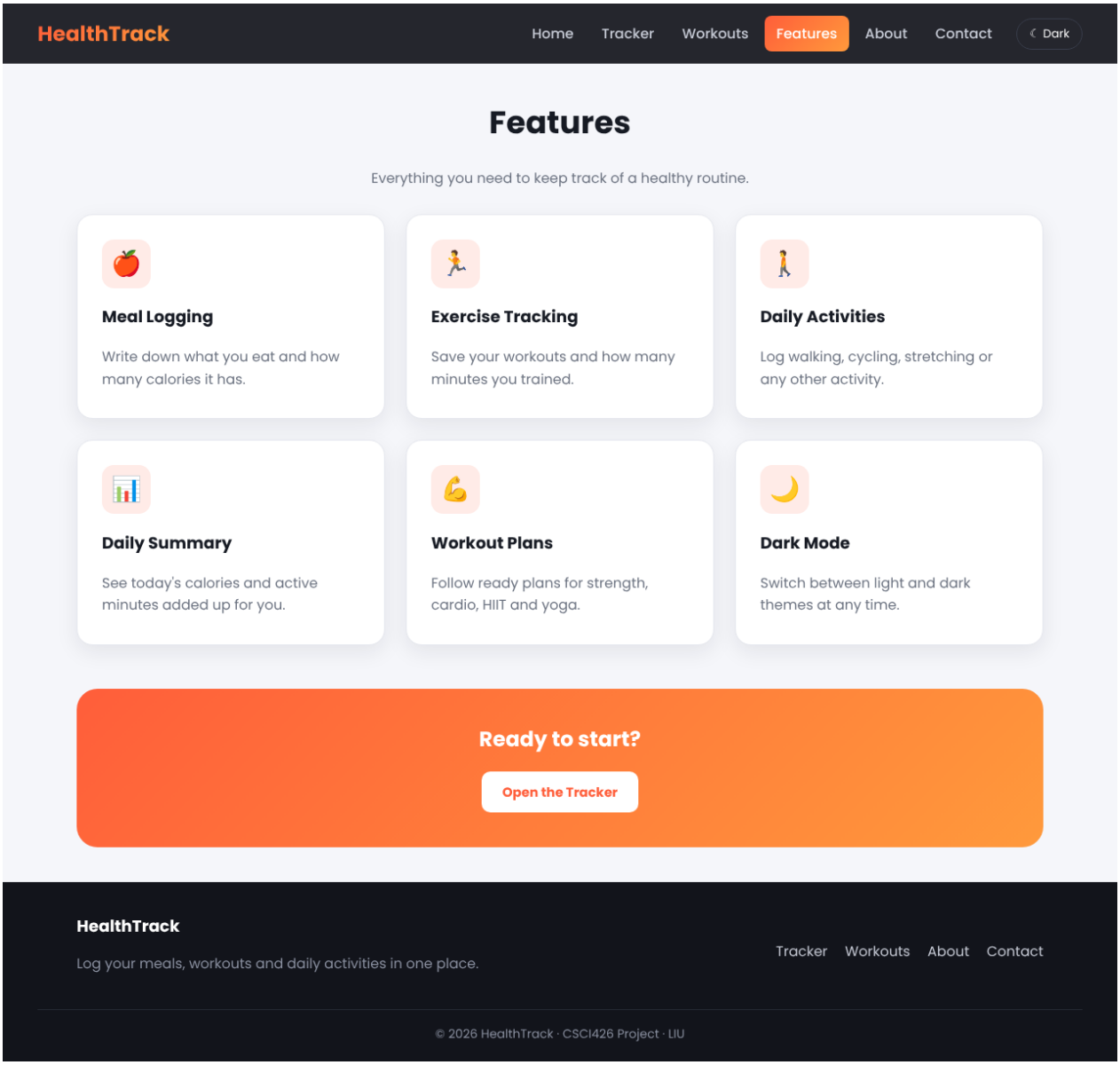


Figure 7 – Features

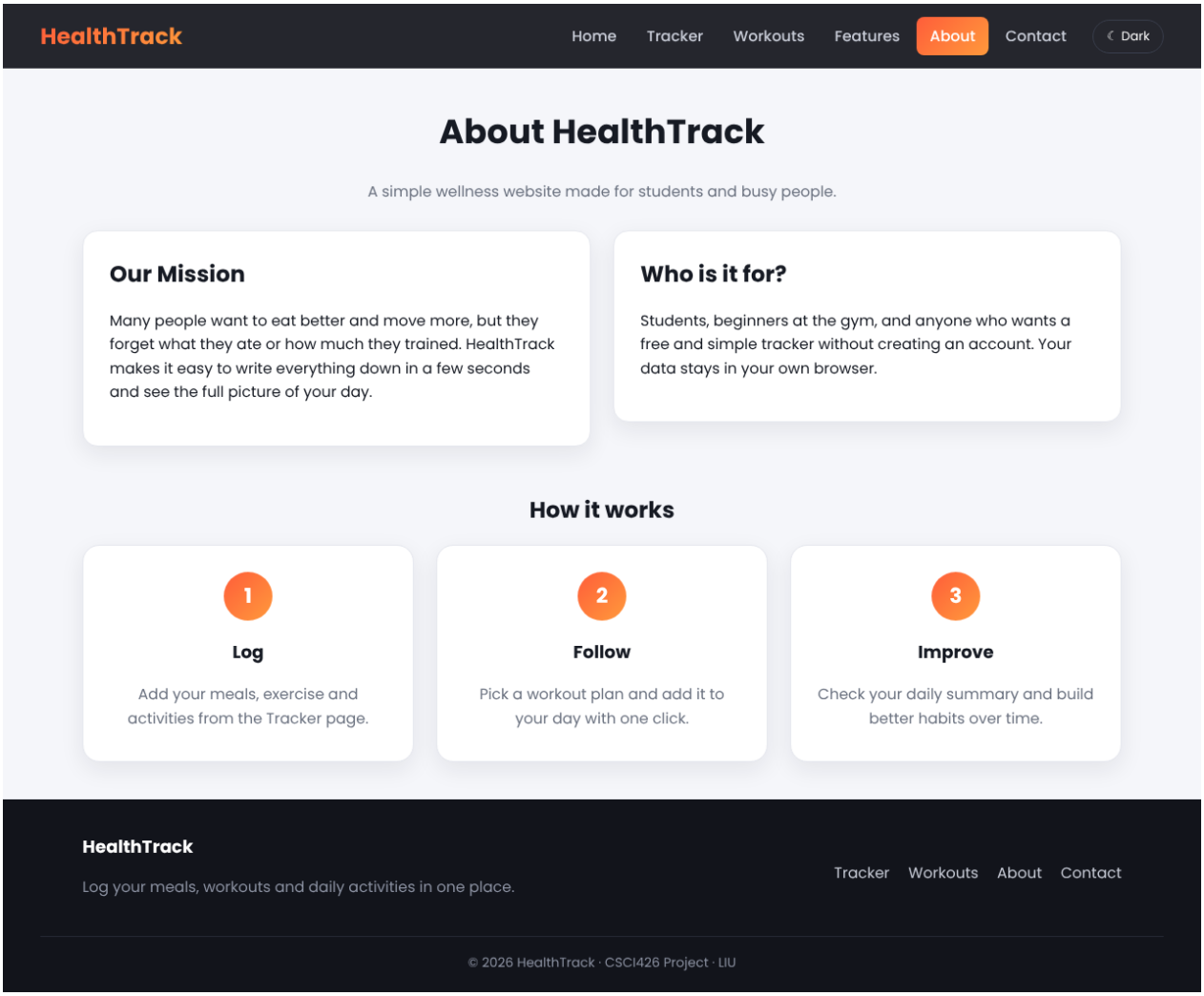


Figure 8 – About

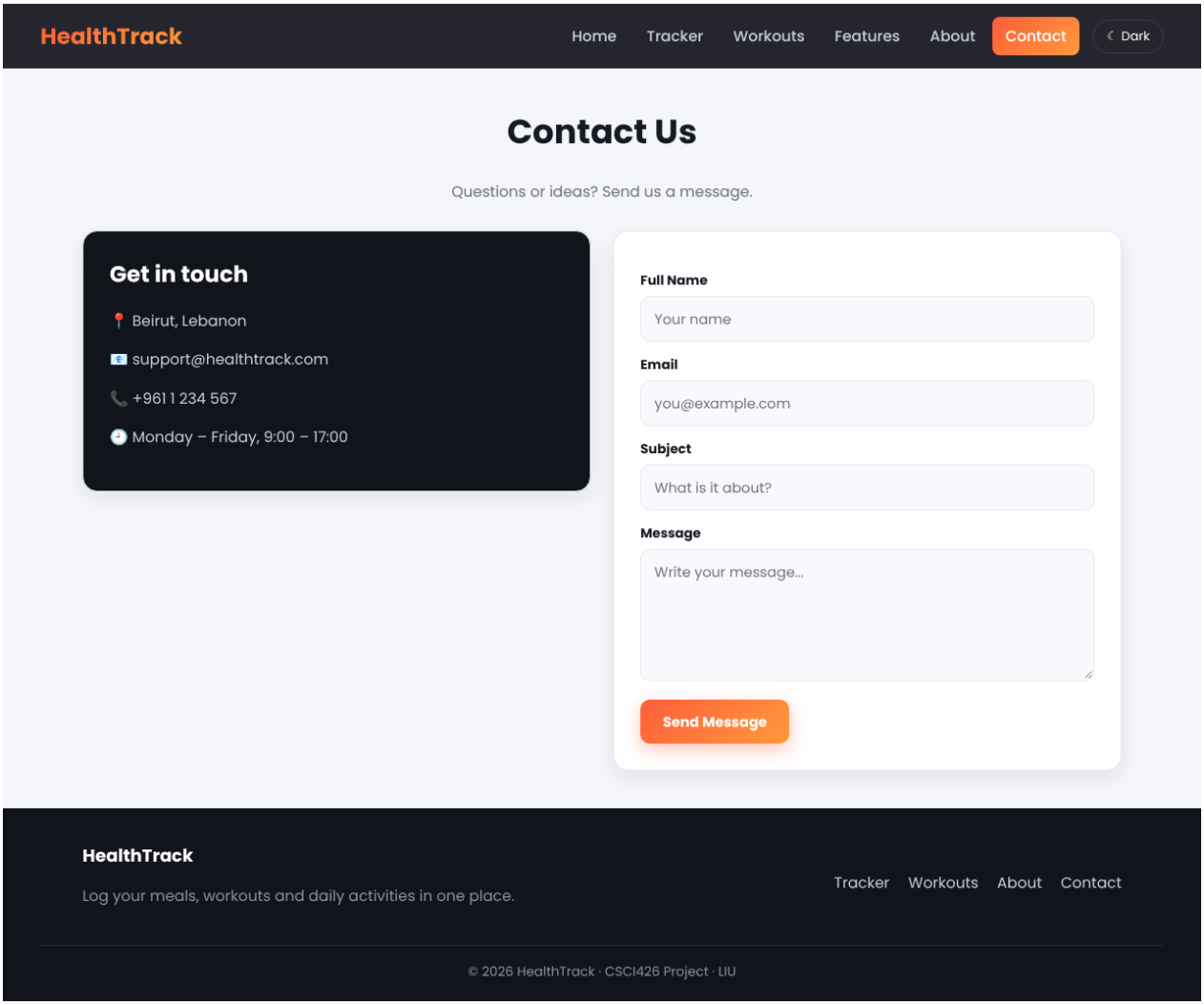


Figure 9 – Contact page

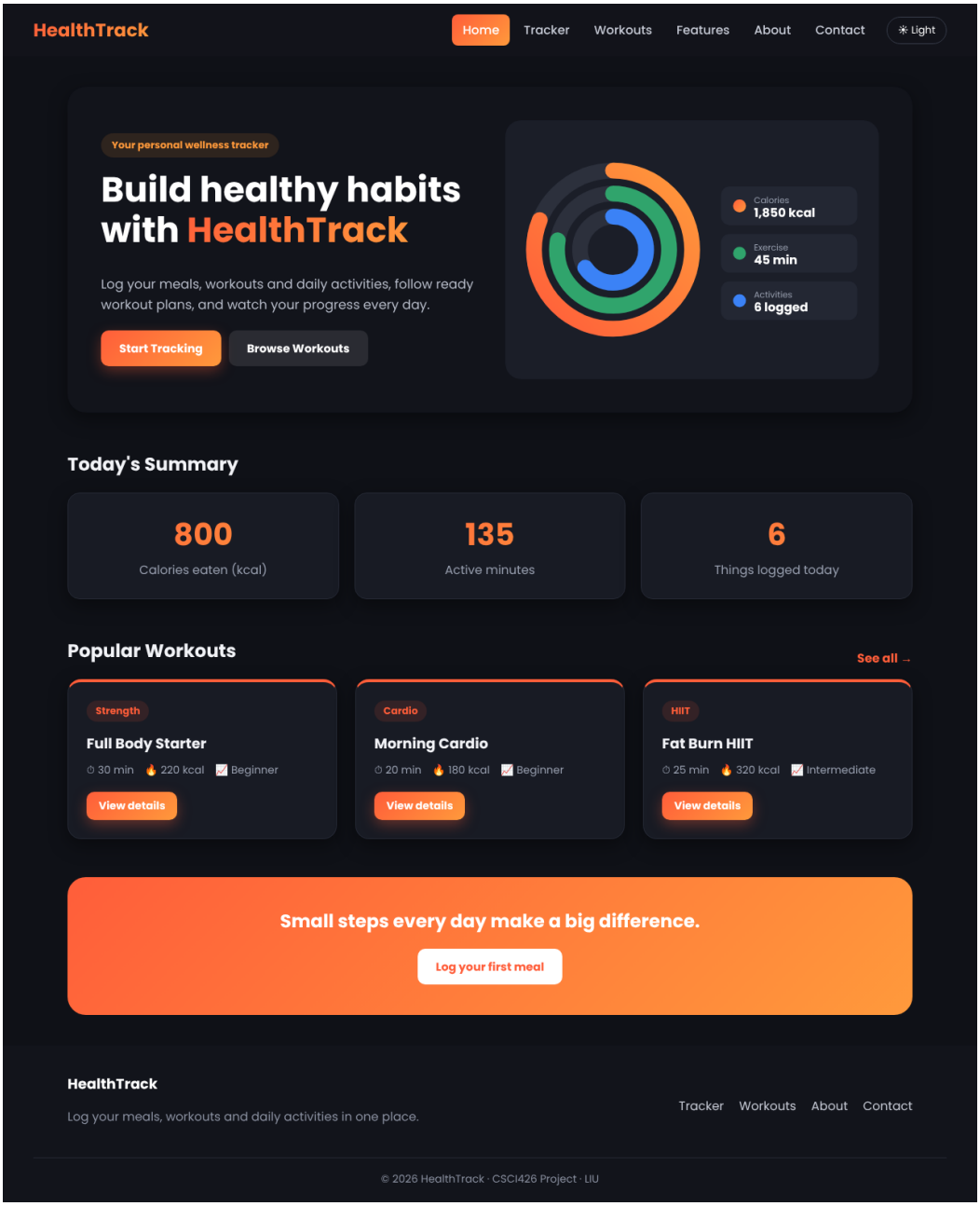


Figure 10 – Dark mode

6. Key Code Snippets

6.1 Routing (App.js)

Each page has its own Route. The `:id` part makes the workout page dynamic, and * catches wrong links. The theme is kept in state and saved with useEffect.

src/App.js

```
const [theme, setTheme] = useState(localStorage.getItem("theme") || "light");

useEffect(() => {
  localStorage.setItem("theme", theme);
}, [theme]);

<div className="App" data-theme={theme}>
  <Router>
    <Navbar theme={theme} setTheme={setTheme} />
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/tracker" element={<Tracker />} />
      <Route path="/workouts" element={<Workouts />} />
      <Route path="/workouts/:id" element={<WorkoutDetails />} />
      <Route path="/features" element={<Features />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </Router>
</div>
```

6.2 Dynamic Page (WorkoutDetails.js)

useParams reads the id from the URL; find returns the matching workout from the JSON file.

src/pages/WorkoutDetails.js

```
const { id } = useParams();            // /workouts/3  ->  id = "3"
const workout = workouts.find((w) => w.id === Number(id));

if (!workout) {
  return <h1>Workout not found</h1>;
}

{workout.exercises.map((ex, i) => (
  <tr key={i}>
    <td>{i + 1}</td>
    <td>{ex.name}</td>
    <td>{ex.sets}</td>
    <td>{ex.reps}</td>
  </tr>
))}
```

6.3 Saving Logs and Validating the Form (Tracker.js)

src/pages/Tracker.js

```
const [entries, setEntries] = useState(
  JSON.parse(localStorage.getItem("entries")) || demoEntries
);

useEffect(() => {
  localStorage.setItem("entries", JSON.stringify(entries));
}, [entries]);

const handleSubmit = (e) => {
  e.preventDefault();
  if (!name || !value || !date) {
    setError("Please fill in all the fields.");
    return;
  }
  if (value <= 0) {
    setError("The number must be bigger than 0.");
    return;
  }
  const newEntry = { id: Date.now(), type, name, value: Number(value), date };
  setEntries([newEntry, ...entries]);
  setSuccess(`${type} "${name}" was added.`);
};

const handleDelete = (id) => {
  setEntries(entries.filter((e) => e.id !== id));
};
```

6.4 Search and Filter (Workouts.js)

src/pages/Workouts.js

```
const shown = workouts.filter((w) => {
  const matchName = w.name.toLowerCase().includes(search.toLowerCase());
  const matchCategory = category === "All" || w.category === category;
  return matchName && matchCategory;
});
```

6.5 Reusable Component with Props (WorkoutCard.js)

src/components/WorkoutCard.js

```
const WorkoutCard = ({ id, name, category, level, minutes, calories }) => {
  return (
    <div className="card workout-card">
      <span className="tag">{category}</span>
      <h3>{name}</h3>
      <div className="workout-info">
        <span>{minutes} min</span>
        <span>{calories} kcal</span>
        <span>{level}</span>
      </div>
      <Link to={`/workouts/${id}`} className="btn btn-small">View details</Link>
    </div>
  );
};
```

6.6 Light and Dark Themes (App.css)

Changing `data-theme` on the main div switches every color at once, because all the components use the same CSS variables.

src/App.css

```
.App[data-theme="light"] { --bg: #f5f6fa; --box: #ffffff; --text: #161a23; }
.App[data-theme="dark"]  { --bg: #0f1117; --box: #181b24; --text: #f2f3f7; }

.App {
  background: var(--bg);
  color: var(--text);
  transition: background 0.3s, color 0.3s;
}
```

7. Responsive Design

The layout uses Flexbox with `flex-wrap` and CSS Grid with `repeat(auto-fill, minmax(260px, 1fr))`, so cards move to new rows automatically on smaller screens. Under 900px a media query hides the navigation links behind a menu button; clicking it toggles a `menuOpen` state in the Navbar.

src/App.css (media query)

```
@media (max-width: 900px) {
  .menu-btn { display: block; }          /* show the menu button*/
  .nav-links { display: none; }          /* hide the links      */
  .nav-links.open { display: flex; }     /* show them when open */
}
```

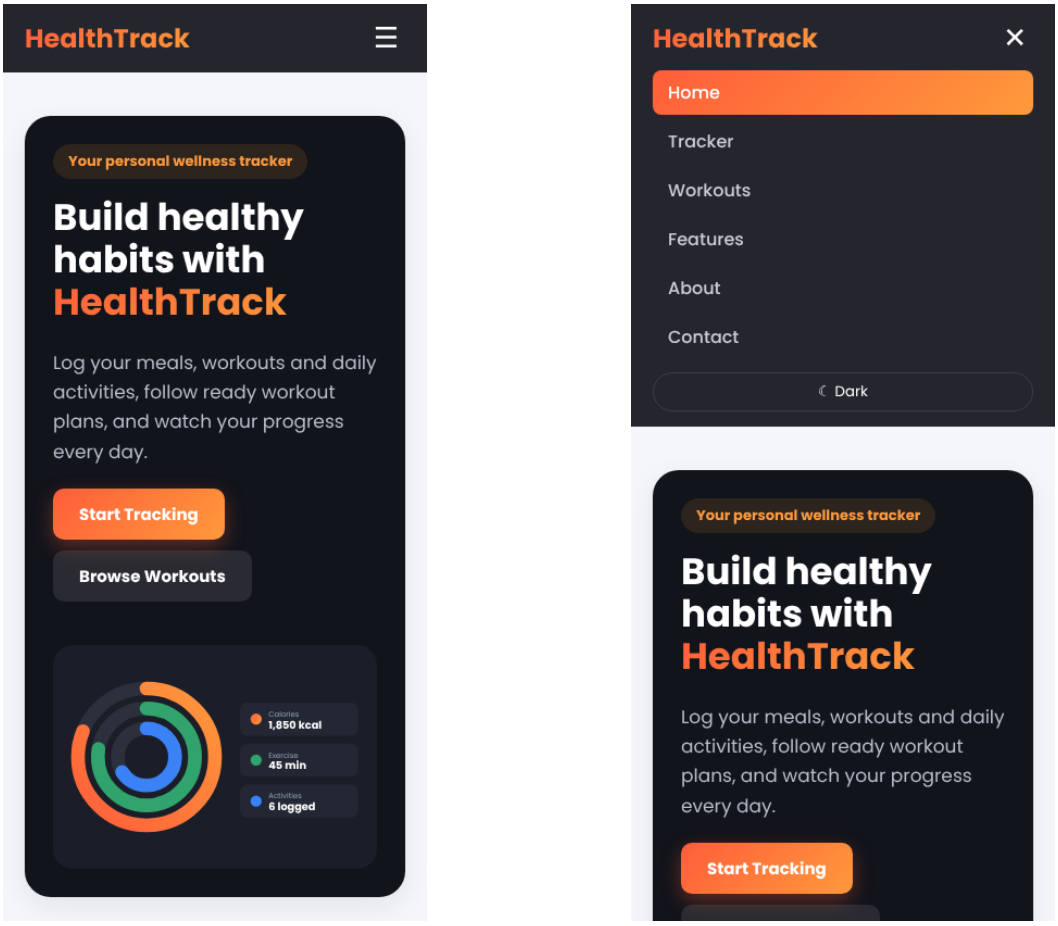


Figure 11 – Mobile view (390px wide): home page and opened menu

8. Version Control and Deployment

- The project is tracked with **Git**, with small commits for each feature (pages, styling, README) so the commit history shows how the project was built.
- The repository is hosted on **GitHub**: `https://github.com/alimoustapha/_____`
- The website is deployed on **Netlify / Vercel**: `https://_____`

- A `_redirects` file (Netlify) and a `vercel.json` file (Vercel) send every URL to index.html, so refreshing a page like /workouts/3 works on the live site.
- The README.md contains the project description, setup instructions and screenshots.

9. Conclusion

HealthTrack meets the project requirements: it is built with ReactJS, has more than five pages including a dynamic page, works on all screen sizes, and is managed with Git and deployed online. Building it practiced the main course topics: components and props, useState and useEffect, React Router with useParams, validated forms, list rendering with map and filter, localStorage, and theming with CSS variables.

Future improvements could include a backend and user accounts, weekly charts of progress, water and sleep tracking, and reminders.